SIMATS ENGINEERING

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	**Assessment:**	Debugging and Optimisation task
Weightage:	45%	**Total Marks:**	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/nonrestoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Identify arithmetic errors and performance bottlenecks in the given problem.
- Analyse the execution steps to locate the source of errors.
- Debug the algorithm by correcting logical and computational faults.
- Optimize the solution using appropriate arithmetic techniques or algorithms.
- Evaluate the optimized solution for correctness, accuracy, and efficiency.

QUESTIONS

1. A computer system performing signed binary subtraction using 2's complement produces incorrect results for certain operand combinations. Debug the subtraction process by examining binary conversion, complement generation, and overflow detection. Suggest appropriate corrections to ensure accurate signed arithmetic.
2. A digital processor implementing a carry look-ahead adder fails to produce the correct sum for some input combinations. Debug the generate and propagate logic to identify the source of the error. Recommend modifications that restore correct operation while maintaining high-speed performance.
3. A hardware multiplier using Booth's algorithm consumes more clock cycles than expected during signed multiplication. Debug the execution sequence by analyzing bit-pair decisions, arithmetic shifts, and register updates. Propose optimization techniques to improve execution efficiency without affecting accuracy.
4. A processor implementing the non-restoring division algorithm produces incorrect quotient values for specific dividend and divisor combinations. Debug the step-by-step division process by examining partial remainders, shifting operations, and quotient-bit generation. Suggest corrections to obtain accurate division results.
5. A graphics processing application experiences performance degradation due to frequent floatingpoint computations using IEEE 754 double-precision representation. Debug the arithmetic operations to identify unnecessary precision or computational overhead. Recommend suitable optimization techniques to improve execution speed while maintaining the required numerical accuracy.

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Identification of Errors / Bugs	20	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

Marks Evaluation:

Q. No	Identification of Errors / Bugs (20)	Debugging Approach & Analysis (20)	Correctness of Debugged Solution (25)	Optimization Techniques Applied (20)	Testing and Documentation (15)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	

3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 20**	**/ 25**	**/ 20**	**/ 15**	**/ 100**

Course Coordinator **Faculty Incharge**

CO 2 ASSESMENT TOOL 2
DEBUGGING & OPTIMISATION TASK

Question 1: Signed Binary Subtraction using 2's Complement

1.Problem Analysis

The processor performs signed subtraction using the **2's complement method**. The subtraction operation is carried out by adding the 2's complement of the subtrahend to the minuend.

$$A - B = A + (2's\ Complement\ of\ B)$$

Incorrect results occur because of:

- Incorrect binary conversion
- Incorrect 2's complement generation
- Carry handling mistakes
- Failure to detect overflow

These issues lead to wrong signed arithmetic results.

2. Identification of Errors/Bugs

- Error 1 – Incorrect Binary Conversion

Decimal operands must first be converted correctly into binary.
Example
15 = 00001111
6 = 00000110
If either number is converted incorrectly, the subtraction result becomes wrong.

- Error 2 – Incorrect 2's Complement

Correct method
Binary of 6
00000110
1's Complement
11111001
Add 1
11111010
If the **+1** step is omitted, subtraction becomes inaccurate.

- Error 3 – Carry Handling

After binary addition, the carry generated beyond the MSB should be discarded.
Keeping the carry changes the final result.

- Error 4 – Overflow Detection

Overflow occurs when

- Positive − Negative produces an invalid negative value.
- Negative − Positive produces an invalid positive value.

Ignoring overflow causes incorrect signed outputs.

3. Debugging Approach & Analysis

Example
A = 15
B = 6

Step 1

Convert into Binary
A = 00001111
B = 00000110

Step 2

Generate 2's Complement

1's Complement
11111001
Add 1
11111010

Step 3

Binary Addition

00001111
+11111010
---
1 00001001
Discard carry
Result
00001001
Decimal = 9
Hence
15 − 6 = 9
Correct.

Overflow Example

A = 127
01111111
B = -2
11111110
Result exceeds positive range.
Overflow Flag = 1
Processor must report overflow.

4. Correctness of Debugged Solution

Correct Algorithm

1. Convert operands into binary.
2. Generate accurate 2's complement.
3. Perform binary addition.
4. Discard final carry.
5. Check overflow flag.
6. Display correct signed result.

5. Optimization Techniques Applied

- Use hardware 2's complement generator.
- Use Carry Look-Ahead Adder.
- Detect overflow using XOR of carry into and carry out of MSB.
- Validate binary inputs before subtraction.
- Reduce unnecessary complement operations.

6. Testing and Validation

Test Case	Expected	Obtained	Status
15−6	9	9	Pass

Test Case	Expected	Obtained	Status
25−12	13	13	Pass
-10−5	-15	-15	Pass
127−(-2)	Overflow	Overflow Detected	Pass
-128−1	Overflow	Overflow Detected	Pass

Question 2: Carry Look-Ahead Adder (CLA)

1. Problem Analysis
The CLA produces incorrect sums because of errors in the Generate (G) and Propagate (P) logic.
2. Identification of Errors/Bugs

- Wrong Generate equation.
- Wrong Propagate equation.
- Incorrect carry calculation.
- Faulty carry connections.

3. Debugging Approach
Example:
A = 1010
B = 0110
Generate carry using:
G = A·B
P = A ⊕ B
Carry:
Ci+1 = Gi + PiCi
Correct Sum = **10000 (16)**
4. Correctness of Solution

- Compute Generate and Propagate correctly.
- Calculate carry in parallel.
- Verify the final carry output.

5. Optimization

- Simplify Boolean expressions.
- Reduce gate delay.
- Use parallel carry computation.

6. Testing

Test	Result
5+4	9 (Pass)
10+6	16 (Pass)
15+1	16 (Pass)

7. Conclusion
Correct Generate and Propagate logic restores accurate high-speed addition.

Question 3: Booth's Multiplication Algorithm

1. Problem Analysis
Booth's Algorithm requires extra clock cycles because of incorrect bit-pair decisions and register updates.
2. Identification of Errors/Bugs

- Wrong bit-pair interpretation.
- Incorrect arithmetic right shift.
- Improper register updates.
- Extra loop iterations.

3. Debugging Approach
Example:
5 × 3
M = 0101
Q = 0011
Check bit pairs:

- 01 → Add
- 10 → Subtract
- 00, 11 → No operation

Perform arithmetic right shift after each step.
Final Product = **15**
4. Correctness of Solution

- Follow correct Booth's rules.
- Update registers after every cycle.
- Perform exactly n iterations.

5. Optimization

- Skip unnecessary operations.
- Use hardware arithmetic shifters.
- Reduce redundant clock cycles.

6. Testing

Test	Result
5×3	15 (Pass)
-5×3	-15 (Pass)
-4×-3	12 (Pass)

Question 4: Non-Restoring Division Algorithm

1. Problem Analysis
The processor produces incorrect quotient values because of errors in remainder calculation and quotient-bit generation.

2. Identification of Errors/Bugs

- Incorrect partial remainder.
- Wrong shift operation.
- Incorrect quotient bit.
- Missing final remainder correction.

3. Debugging Approach
Example:
13 ÷ 3
Dividend = 1101
Divisor = 0011
Perform left shift, update remainder, and generate quotient bits.
Final Result:
Quotient = **4**
Remainder = **1**
4. Correctness of Solution

- Shift correctly.
- Update remainder properly.
- Generate quotient bits accurately.
- Correct the final remainder if needed.

5. Optimization

- Use fast shift registers.
- Simplify remainder comparison.
- Reduce control delay.

6. Testing

Test	Result
13÷3	4 R1 (Pass)
15÷5	3 R0 (Pass)
30÷5	6 R0 (Pass)

Question 5: IEEE 754 Double-Precision Floating-Point Optimization

1. Problem Analysis
A graphics processing application experiences slow performance due to frequent IEEE 754 double-precision floating-point computations. The processor uses more precision than necessary, increasing computation time.
2. Identification of Errors/Bugs

- Unnecessary use of double precision for simple calculations.
- Repeated floating-point operations.
- Excessive rounding and normalization.
- High computational overhead.

3. Debugging Approach
Example:
Numbers:
A = **12.5**
B = **3.25**
IEEE 754 performs:

1. Align exponents.
2. Add mantissas.
3. Normalize the result.
4. Round the mantissa.

These steps are repeated for every floating-point operation, causing execution delays.
4. Correctness of Solution

- Use double precision only when high accuracy is required.
- Reduce unnecessary floating-point calculations.
- Optimize arithmetic operations.
- Verify the numerical accuracy after optimization.

5. Optimization Techniques

- Use **single precision** instead of double precision where possible.
- Minimize repeated calculations.
- Use hardware Floating-Point Unit (FPU).
- Apply compiler optimization techniques.
- Store frequently used values to avoid recomputation.

6. Testing

Test Case	Expected Result	Status
12.5 + 3.25	15.75	Pass
20.5 − 5.5	15.0	Pass
8.5 × 2	17.0	Pass